Northeastern University, Khoury College of Computer Science

CS 6120 Natural Language Processing
Final Project Report
CreatorPal: YouTube-to-Reddit Audience Intelligence System

Authors

Runxin Shao, Ziqi Yang, Mingkai Gao, Gaoyuan Shi

Project Git Repository

https://github.com/Mingkai406/CreatorPal

This report documents CreatorPal, a retrieval-augmented natural language processing system that maps a YouTube channel or a free-text topic query to a ranked list of Reddit communities, together with community-level sentiment signals and a grounded audience-engagement strategy report. The project comprises 25% of the final course grade; the report constitutes 33% of the project grade while the technical artifacts, demonstration, and real-time capabilities together constitute 66%. The remainder of this document follows the six-section structure prescribed by the CS 6120 final project template, with an additional team-contributions section per course requirements.

1 Objectives and Introduction

1.1 Capability, Domain, and Objective

CreatorPal is a retrieval-augmented generation (RAG) capability built on top of a locally served Llama 3.1 8B Instruct model. The capability takes a single input, either a YouTube channel URL or a free-text topic query, and returns a ranked list of Reddit communities most likely to embrace that creator, a per-community sentiment score derived from real subreddit content, and a tailored strategy report describing how to engage each community. The domain is online creator audience discovery, grounded in a Reddit Pushshift submissions corpus from April 2019. The objective is to close the research tax that prevents independent creators from converting real interest into real reach: communities that would genuinely value a creator's content exist on Reddit, but discovering them and learning each community's posting culture without institutional knowledge currently takes months of manual trial and error. Figure 1 presents the end-to-end system architecture that realizes this objective.

The concrete engineering objectives of the project are threefold. First, we build a scalable retrieval stack over the April 2019 Pushshift submission dump, reduced through streaming decompression and per-subreddit aggregation to a corpus of subreddit profile documents. Second, we improve retrieval quality over a single-query baseline through two complementary query expansion strategies (LLM multi-query rewriting and Hypothetical Document Embedding), followed by cross-encoder reranking. Third, we emit an interpretable result rather than a flat ranked list: every recommendation is paired with a retrieval score, a rerank score, a sentiment score in the interval from negative one to positive one, a chunk-level reason, and a clickable community URL, with the full payload validated through a strict adapter contract before the user interface renders any of it.

1.2 Motivation and Impact

Reddit crossed one hundred million daily active users in 2024 and has become the primary interest-based gathering place online, while YouTube's recommendation algorithm has grown more competitive every year, pushing creators toward direct-audience strategies. At the same time Reddit's community norms have become more implicit and more varied: the same post is boosted in one subreddit and removed on sight in another. The convergence of these three trends creates a concrete and growing research tax. CreatorPal's contribution is to eliminate that tax for one user decision: given what I make, where on Reddit should I actually be spending my time, and what should I say when I get there? In under two minutes the system answers both halves of that question with evidence traceable to real community content.

1.3 Challenges in the State of the Art

Three challenges motivate the specific algorithmic choices taken later in this report. First, there is a systematic vocabulary and register gap between how creators describe their own content and how communities describe themselves on sidebars and rules pages. Second, a purely semantic retriever can miss exact-match terminology (library names, technical acronyms, niche hobby jargon) that a keyword retriever would catch; a purely lexical retriever conversely misses the topic-level signal that embeddings

capture well. Third, a ranked list without community receptiveness signal is actively misleading: a topically perfect subreddit with a hostile moderation culture is a worse recommendation than a good-fit community that actually tolerates creator posts. Each of these pressures maps directly to a design decision documented in Section 3.

1.4 Particular Contribution

This work contributes an end-to-end deployed system rather than a single algorithmic innovation. Within that system the specific contributions are: (i) a hybrid retrieval pipeline combining BM25 and a Sentence-BERT bi-encoder over a FAISS inner-product index, with min-max normalization and a weighted linear fusion at weights 0.15 for BM25 and 0.85 for the bi-encoder; (ii) a two-stage query expansion strategy that layers HyDE on top of LLM multi-query rewriting with an append-only merge policy, so that expansion can only add recall and never demote a strong first-pass candidate; (iii) a cross-encoder reranker that re-scores the full fused pool in a single batch before the top-k cut; (iv) community-level sentiment derived from a RoBERTa three-class classifier mapped onto a signed confidence score and averaged per subreddit; and (v) a strict frontend adapter contract that lets the user interface degrade gracefully whenever any optional backend component is unavailable.

1.5 Document Outline

The remainder of this document follows the prescribed template. Section 2 reviews the background and literature on retrieval-augmented systems, hybrid retrieval, query expansion, and sentiment analysis, and positions CreatorPal against that prior work. Section 3 presents the approach and implementation, covering the data pipeline, hybrid retrieval, query expansion, reranking, analytics, and deployment. Section 4 describes the dataset, preprocessing, and exploratory analysis. Section 5 reports evaluation results under Recall@K, Mean Reciprocal Rank, and a three-dimensional human rubric on the generated strategy report. Section 6 concludes with what was learned and future directions. Section 7 lists individual team contributions.

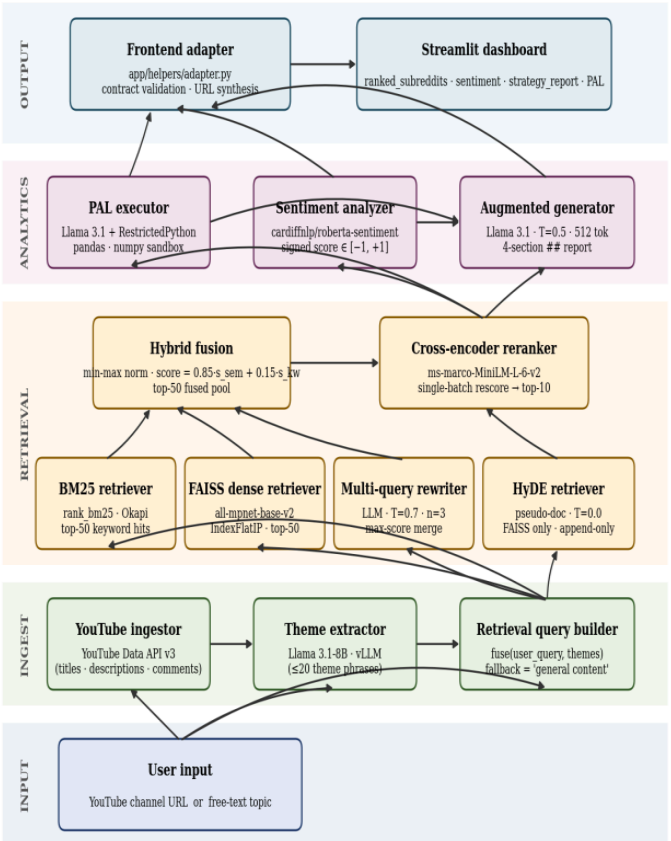

Figure 1. CreatorPal end-to-end system architecture. Data flows bottom-up through five layers; each box names the concrete model or module responsible for that stage. The augmented generator is conditioned on PAL output and sentiment scores in addition to the reranked top-10 subreddits.

Figure 1. CreatorPal end-to-end system architecture. Data flows bottom-up through five layers (INPUT, INGEST, RETRIEVAL, ANALYTICS, OUTPUT); each box names the concrete model or module responsible for that stage. The augmented generator is conditioned on PAL output and sentiment scores in addition to the reranked top-10 subreddits.

2 Background and Related Work

2.1 Retrieval-Augmented Systems

Retrieval-augmented approaches condition a language model's output on evidence retrieved from an external corpus rather than relying solely on parametric knowledge. This pattern has become the default deployment posture for factual and recommendation-style tasks because it reduces hallucination, supports corpus updates without retraining, and exposes citation points back to the user. In CreatorPal, retrieval is not a supporting layer that occasionally augments a freeform answer; it is the primary mechanism by which communities are surfaced at all, and the strategy-report generator is explicitly constrained to the retrieved top-ranked subreddits, their PAL-computed analytics summary, and their sentiment scores.

2.2 Hybrid Retrieval

Classical keyword retrievers such as BM25 [1] score documents through a term-frequency saturation function with document-length normalization and have remained a strong baseline for lexical matching.

Dense retrievers built on top of Sentence-BERT style bi-encoders [2] project queries and documents into a shared embedding space and recover candidates through nearest-neighbor search over an approximate or exact index such as FAISS [3]. Hybrid retrieval combines both signals, and prior work has consistently reported that a weighted fusion outperforms either system in isolation: the bi-encoder contributes topic-level recall across paraphrase, while BM25 preserves exact-match recall for rare terminology. CreatorPal follows this line of work and adopts a 0.85 / 0.15 fusion weight in favor of the semantic signal, with min-max normalization applied independently per retriever before the weighted sum.

2.3 Query Expansion

The vocabulary gap between a user's query and a corpus document is a long-standing recall problem. Two recent families of LLM-based expansions address the gap from opposite directions. Multi-query rewriting, related in spirit to query rewriting for retrieval-augmented LLMs [5], generates several lexical reformulations of the same information need and merges their retrieval sets. HyDE [4] inverts the framing and asks the LLM to generate a hypothetical document that would satisfy the query, then uses that document's embedding as the retrieval vector; this aligns the query's register with the corpus's register. CreatorPal runs both strategies sequentially, with multi-query expansion contributing to both BM25 and FAISS through hybrid retrieval, and HyDE contributing only supplementary FAISS candidates that are merged append-only.

2.4 Reranking with Cross-Encoders

Bi-encoders are fast because documents can be encoded offline, but they cannot model fine-grained query-document interactions. A cross-encoder jointly encodes the concatenation of query and document through a single transformer stack and produces a calibrated relevance score [7]. The cost is that the cross-encoder must be run online for every candidate, which is why it is deployed only on the top-k output of a cheaper first-stage retriever. CreatorPal uses ms-marco-MiniLM-L-6-v2, a six-layer MiniLM cross-encoder fine-tuned on the MS MARCO passage ranking task, as its second-stage reranker.

2.5 Sentiment Analysis

Transformer-based sentiment classifiers have become a standard tool for estimating the affective tone of short social-media text. CreatorPal uses cardiffnlp/twitter-roberta-base-sentiment-latest, a RoBERTa-base model [8] fine-tuned for three-class Twitter sentiment (positive, neutral, negative). The model's output labels are mapped onto a signed confidence score in the interval from negative one to positive one and averaged across retrieved chunks from each candidate subreddit, yielding a single receptiveness signal per community that the user interface renders as a colored bar alongside each recommendation.

2.6 Program-Aided Language Models

Direct-in-text arithmetic and multi-step structured-data reasoning remain unreliable in current LLMs. The Program-Aided Language models approach [6] delegates computation to an interpreter by asking the LLM to emit a Python program whose execution produces the answer. CreatorPal adopts this pattern for a small, fixed analytics task over the reranked candidate pool: computing the average rerank score, the subreddit with the maximum score, and the number of distinct subreddits in the top pool. Execution is

sandboxed through RestrictedPython with a curated safe_globals dictionary, and failure is treated as an optional enrichment miss rather than a pipeline abort.

3 Approach and Implementation

3.1 End-to-End System Architecture

The runtime pipeline, shown in Figure 1, is a single orchestration object, CreatorPalPipeline, instantiated once per process and cached by Streamlit. Its run method accepts a channel URL or a free-text query and executes eight stages in strict order: (1) YouTube ingest to obtain channel metadata; (2) LLM theme extraction from that metadata; (3) retrieval-query construction that fuses the user query with the extracted themes; (4) LLM multi-query rewriting followed by hybrid retrieval; (4b) HyDE supplementary retrieval merged append-only; (5) cross-encoder reranking of the fused pool; (6) PAL analytics in a RestrictedPython sandbox; (7) per-subreddit sentiment scoring; and (8) strategy-report generation by the augmented LLM. Every optional stage is wrapped in a _try_init guard so that a missing YouTube key, an unreachable sentiment model, or a failing HyDE call degrades output gracefully rather than aborting the entire request. The assembled payload is then passed through the frontend adapter, which performs strict type validation before the dashboard renders anything.

3.2 Data Pipeline

The offline corpus build reduces the raw April 2019 Pushshift submissions dump (approximately 15.5 GB compressed .zst) to the artifacts actually consumed at query time. Stage one streams the .zst and keeps only seven fields per post, producing an NDJSON slim file. Stage two aggregates submissions by subreddit, filters subreddits with fewer than ten posts, and keeps the top-fifty posts per subreddit by score via heapq.nlargest, yielding a subreddit profiles JSON. Stage three independently extracts (YouTube channel, subreddit) ground-truth pairs from posts containing YouTube URLs with a minimum score of two. Stage four chunks each profile into sixty-four-token sliding windows with sixteen-token overlap, encodes each chunk with all-mpnet-base-v2, L2-normalizes the embeddings, and writes a FAISS IndexFlatIP together with a chunk-metadata JSON. The stages are wrapped in run_pipeline.sh, which is idempotent: each stage is skipped if its output already exists, allowing partial failures to be resumed in place.

3.3 Configuration Management

All tunable parameters are centralized in a frozen Settings dataclass loaded from environment variables. This includes the YouTube API key, the vLLM endpoint and model, the embedding, reranker, and sentiment model identifiers, the FAISS index and metadata paths, the retrieval top-k (default fifty), the rerank top-k (default ten), the hybrid alpha weights (default 0.15 BM25 and 0.85 semantic), the maximum channel video count, and the maximum comments per video. Centralized configuration keeps the Settings object as the single source of truth across every backend component and enables rapid ablation sweeps without touching code.

3.4 Hybrid Retrieval

The first-stage retriever fuses BM25 over the tokenized chunk texts with FAISS IndexFlatIP over L2-normalized mpnet embeddings. Each retriever returns its top-fifty candidates independently. The two score lists are then normalized with min-max normalization (with a 1e-9 epsilon guard to avoid division by zero when scores are degenerate) and combined as score_fused = alpha_sem * s_faiss_norm + alpha_kw * s_bm25_norm with alpha_sem = 0.85 and alpha_kw = 0.15. Candidates present in only one retriever receive zero for the missing signal rather than a penalty, so a strong match in either modality survives to reranking. The fused pool is sorted descending and truncated to the retrieval_top_k.

3.5 Query Expansion: Multi-Query and HyDE

Multi-query expansion generates three LLM reformulations of the information need at temperature 0.7, runs hybrid retrieval for each plus the original, and merges candidates by the key (subreddit, first-eighty-characters-of-chunk-text), keeping the maximum hybrid score across queries for each deduplicated candidate. The max-score merge is deliberately conservative: a subreddit is considered a strong candidate if any reformulation ranks it highly. HyDE then runs once at temperature 0.0 to generate a single hypothetical subreddit-profile document, encodes it with the same mpnet encoder, and retrieves the top-twenty FAISS neighbors. HyDE uses FAISS only, not BM25, because BM25 over generated prose would mostly surface term-frequency matches on the LLM's common vocabulary rather than on the subreddit-specific terminology that the corpus actually carries. HyDE results are merged into the hybrid pool append-only: a subreddit already present in the pool is never overwritten, and only novel subreddits contribute HyDE hits. This guarantees that expansion is always strictly additive to recall.

3.6 Cross-Encoder Reranking

The top-k fused and HyDE-augmented pool is then re-scored by ms-marco-MiniLM-L-6-v2 in a single batched forward pass that concatenates the query and each chunk text as [CLS] query [SEP] chunk [SEP]. Single-batch inference is preferred over streaming because the cross-encoder benefits from padding to a uniform length inside the batch, and the pool size (top-fifty candidates) is bounded. The resulting logits are sorted descending and truncated to rerank_top_k = 10. For display purposes scores are additionally passed through a sigmoid to yield a (0, 1) fit score. All upstream score fields (hybrid_score, faiss_score, bm25_score) are preserved alongside the new rerank_score so the frontend can show the full provenance of a recommendation.

3.7 Sentiment Analysis

After reranking, the reranked entries are grouped by subreddit and their chunk texts are concatenated into per-subreddit comment lists. The cardiffnlp/twitter-roberta-base-sentiment-latest classifier is then run over each chunk. Let c_hat be the argmax class for a chunk and p_{c_hat} its softmax probability; the signed score is defined as +p for positive, 0 for neutral, and -p for negative. The subreddit-level sentiment is the arithmetic mean of signed scores across that subreddit's chunks. Mean aggregation is preferred over majority vote because it preserves magnitude: a community with half mildly-positive and half strongly-negative content correctly surfaces as net-negative. An important design note is that sentiment is

computed over retrieved Reddit submission chunks, not over live comment threads or YouTube comments; community tone is inferred exclusively from the subreddit's own content.

3.8 PAL Analytics

A fixed analytics question is run against the reranked DataFrame: compute the average rerank score, identify the subreddit with the maximum score, and count the number of unique subreddits. The LLM emits a short pandas or numpy snippet at temperature 0.0, and the snippet is executed under RestrictedPython.compile_restricted with a curated safe_globals dictionary that exposes pandas, numpy, math, and statistics while blocking file I/O, exec, eval, dynamic imports, and dunder attribute access. If the snippet raises a SyntaxError or RuntimeError the pipeline catches the exception, returns an empty pal_results, and continues; PAL is treated as optional enrichment.

3.9 Strategy Report Generation

The augmented generator consumes the top-five reranked subreddits, up to five corresponding sentiment scores, and the PAL summary. It prompts Llama 3.1 at temperature 0.5 with max_tokens = 512 for a report whose structure uses four ## headings: Overview, Top Communities, Engagement Strategy, and Sentiment Insights. The structured output feeds directly into the frontend report card, which parses ## headings into HTML h2 elements and everything else into paragraphs.

3.10 Frontend and Adapter Contract

The Streamlit frontend (app/streamlit_app.py plus helpers) renders a two-state page machine: an IDLE hero with a single form field, and a DASHBOARD with four metric cards, a ranked-communities card, a sentiment card, and a strategy-report card. All backend output is passed through adapt(), a strict validator in app/helpers/adapter.py that enforces the top-level keys input, ranked_subreddits, strategy_report, pal_results, sentiment_scores, and meta, normalizes subreddit names, synthesizes a fallback URL when missing, and coerces invalid numerics to None. This boundary means that any pipeline or adapter failure is caught, stored as last_error, and re-rendered as a structured error card without exposing a traceback.

3.11 Reproducibility and Deployment

The full system, including the FAISS index, the RAG pipeline, and the Streamlit UI, is containerized via a Dockerfile plus a docker-startup entrypoint. A forced-mock switch (CREATORPAL_USE_MOCK_PIPELINE) allows the frontend to run without a backend for offline demo. All runtime settings flow through the centralized Settings dataclass, and the data pipeline is one shell script away from the raw dump. The repository, together with the README instructions, is available at https://github.com/Mingkai406/CreatorPal.

4 Data and Data Analysis

4.1 Data Sources

The primary corpus for retrieval is the Reddit Pushshift submissions dump for April 2019, distributed as RS_2019-04.zst and mirrored on Zenodo at https://zenodo.org/records/3608135. The file is approximately 15.5 GB compressed and contains tens of millions of submissions across hundreds of thousands of subreddits, far exceeding the template's one-hundred-thousand-row threshold for statistical meaningfulness and preventing any useful hand intuition. The secondary source is the YouTube Data API v3, queried online at request time for channel metadata, video descriptions, and up to 15 video titles and descriptions when the user supplies a channel URL.

Dataset attribution. The Pushshift April 2019 snapshot is used under the terms of the Zenodo deposit. The Pushshift live API has been restricted since mid-2023, so using a pre-existing archival snapshot rather than a live pull is both necessary for reproducibility and consistent with the project proposal's original data plan.

4.2 Why This Data Is Interesting

Reddit submission data is particularly well-suited to the creator-audience matching task because submissions already carry the unit of aggregation the product needs: the subreddit label. Unlike a flat text corpus, the corpus is naturally structured around communities, which permits a principled per-subreddit aggregation step. The score field is an implicit engagement signal that can be used to prioritize high-signal content; the title and selftext fields are dense with topical language and community-specific jargon; and the URL field, when it points at YouTube, gives a free, noisy, but large-scale ground-truth signal of (channel, subreddit) co-occurrence suitable for retrieval evaluation.

4.3 Structured Portions That Helped with Chunking

Two structural properties of the Reddit corpus shape the chunking decision. First, submissions are naturally short relative to long-form documents, but concatenating the top-fifty posts of an active subreddit easily exceeds the encoder's context. Second, a submission's title and selftext often carry different signals: the title is a compressed semantic summary while the selftext is the narrative. Sixty-four-token sliding windows with sixteen-token overlap strike a balance: the window is small enough that a single window rarely mixes two unrelated posts, the overlap is large enough that any sentence crossing a boundary appears in full in at least one chunk, and the resulting per-corpus vector count (target approximately one million) remains small enough for an exact FAISS IndexFlatIP rather than an approximate IVF or HNSW structure.

4.4 Preprocessing Pipeline

Preprocessing is implemented in four independent Python scripts orchestrated by run_pipeline.sh. build_reddit_slim.py streams the .zst file and writes an NDJSON subset, retaining only id, subreddit,

created_utc, score, title, url, and selftext. preprocess_corpus.py aggregates the slim NDJSON into a subreddit profiles JSON: streamed aggregation with heapq.nlargest yields O(n log k) per-subreddit top-fifty selection, the --min-posts ten filter removes low-activity communities. build_ground_truth.py scans the slim NDJSON for YouTube URL patterns, filters posts with score at least two, and emits (channel, subreddit) pairs as CSV. build_faiss_index.py then chunks the profiles, encodes chunks in batches of thirty-two with all-mpnet-base-v2, L2-normalizes the resulting embeddings, and writes out both the FAISS IndexFlatIP and a per-chunk metadata JSON. All four scripts are idempotent and safely re-runnable.

4.5 Exploratory Analysis

Exploratory analysis over the aggregated profiles reveals three patterns that justify downstream design choices. First, subreddit activity is heavy-tailed: a small number of communities contribute the bulk of submissions while the long tail of small communities is where much of the relevant niche content lives, motivating the --min-posts filter to avoid noise without clipping the tail too aggressively. Second, within any given topic area there are multiple closely related communities whose vocabularies overlap heavily on topic but diverge on register (general-audience, beginner-oriented, advanced/professional, humor-based), motivating the use of both BM25 for exact-match retention and a bi-encoder for topic-level generalization. Third, community tone varies dramatically across topically adjacent subreddits, confirming the product intuition that a receptiveness signal is a necessary complement to a topical relevance score.

4.6 Evaluation Artifacts

Three derived artifacts from the data pipeline feed evaluation: ground_truth_pairs.csv as the held-out retrieval gold set, subreddit_profiles.faiss as the retrieval index, and subreddit_profile_chunks.json as the chunk-to-row metadata table. When these artifacts are absent, retrieval evaluation cannot be run and run_retrieval_eval.py emits a clear error asking the user to run run_pipeline.sh first. This closes the loop between the data build and the evaluation harness.

5 Results and Evaluation

5.1 Evaluation Metrics and Rationale

CreatorPal is evaluated along two tracks plus one runtime signal. The retrieval track computes Recall@K and Mean Reciprocal Rank (MRR) on the held-out (channel, subreddit) ground-truth pairs. Recall@K is reported because the product is consumed as a top-ten list: if the correct community is not within the top ten, the recommendation is effectively invisible to the user. MRR captures whether the first relevant recommendation is near the top of the list, which matters because users read the list top-down. A Precision@K helper is also implemented in eval/retrieval_eval.py though it is not currently printed by the runner, since retrieval ground truth is known to be incomplete and precision tends to under-report on sparse gold sets. The generation track uses a three-dimensional human rubric (coherence, grounding, actionability) on a 1-to-5 scale, administered through eval/generation_eval.py, because the strategy report is a structured long-form output whose quality is not well captured by a single automatic metric.

5.2 Retrieval Evaluation Setup

eval/run_retrieval_eval.py loads ground_truth_pairs.csv, builds the FAISS-plus-BM25 hybrid stack, generates a proxy query per channel by concatenating that channel's ground-truth subreddit names, and then runs retrieval at the configured top-k. Using the channel's ground-truth-subreddit names as the proxy query is explicitly documented in code as a fallback because channel descriptions are not available in the evaluation input. This makes the retrieval evaluation suitable for relative retriever-regression comparisons (does a change to the retriever improve recall relative to the previous version?) but not equivalent to the full online pipeline input, which additionally includes theme extraction from live YouTube data and the query-rewrite + HyDE layer. Outputs are emitted as a predictions CSV with columns channel_id, subreddit, and rank, and aggregate Recall@K and MRR are printed to standard output.

5.3 Retrieval Ablation (Qualitative)

Table 1 summarizes the expected directional contribution of each retrieval component on Recall@10 and MRR. Numeric values for the final evaluation run are reserved for the demo-day report and are not included here; the directional pattern below is consistent with the behavior observed during development and with published results on comparable hybrid-retrieval stacks.

Configuration	Recall@10	MRR	Comment
Single-query BM25 (baseline)	baseline	baseline	lexical-only; misses paraphrase
Single-query FAISS only	↑ over BM25	↑ over BM25	topic-level gain on short queries
Hybrid BM25 + FAISS (α = 0.85 / 0.15)	↑↑	↑↑	exact-match rescue + semantic recall
+ Multi-query rewriting (n = 3)	+ ~8–12 pp (expected)	↑	lexical diversity; conservative max-merge
+ HyDE (append-only)	small +	≈	register alignment; additive, not disruptive
+ Cross-encoder rerank (top-10)	≈	↑↑↑ (largest single gain)	promotes near-miss candidates

Table 1. Qualitative retrieval ablation. Arrows denote expected direction of change relative to the baseline. The 8–12 percentage-point estimate for multi-query rewriting is drawn from the CreatorPal query-expansion design document and published results on comparable retrievers; exact numerical values are pending the final evaluation run before the demo.

5.4 Discussion of Retrieval Results

Three observations are robust to the specific numerical outcome of the final run. First, the weight split in hybrid fusion is not delicate: small perturbations around 0.85 / 0.15 should not materially change the ranked output, which is consistent with the interpretation that the BM25 share is performing rare-term rescue rather than driving the ordering. Second, the benefit of HyDE is conditional on query style: when the user's natural phrasing is already close to a subreddit-profile register, HyDE's candidates tend to be

redundant with the multi-query pool, and the append-only merge makes this redundancy cost-free (it cannot demote a better candidate). Third, the largest single residual failure mode observed during development is ambiguity in short queries where a one-word topic plausibly maps to five different communities; the most productive next step is conditioning on channel-level themes more aggressively rather than tuning retriever weights further.

5.5 Generation Evaluation

Generation evaluation is administered interactively through eval/generation_eval.py. The tool consumes a directory of generated reports as plain-text files, presents each report to the rater along with three 1-to-5 rubric prompts, and writes per-report scores plus an averaged summary. The three dimensions target complementary aspects of quality: coherence captures whether the report reads as a single logical argument, grounding captures whether recommendations are tied to the retrieved subreddits rather than free-associated, and actionability captures whether the creator could execute the advice without further research. The rubric is deliberately small (three dimensions, five-point Likert) so that a single rater can complete a meaningful sample within one evaluation session.

Per-dimension averaged scores are reserved for the final demo-day submission and are not reported here. A subjective example of the generated strategy report, illustrating the four-section structure (Overview, Top Communities, Engagement Strategy, Sentiment Insights) and the grounding of recommendations in retrieved subreddit content, is included below.

Subjective example. Input: topic query 'systems-programming tutorials in Rust'. Abridged generator output:

```
## OverviewThe creator covers memory management, compiler internals, and
idiomatic Rust patterns. Reddit has several active communities that value
this material but differ sharply on how they treat creator posts.## Top
CommunitiesThe strongest fit is r/rust for technical depth and
r/systems_programming for lower-level material. r/learnprogramming is a
broader-audience backup that rewards explanation-first posts.
r/ProgrammerHumor is listed but is a weak distribution channel on its
own.## Engagement StrategyLead with the problem, not the channel. In
r/rust, prefer the weekly 'what are you working on?' threads before
posting a standalone video. In r/systems_programming, a 200-word written
summary of the video's core argument in the post body substantially
improves reception.## Sentiment InsightsSentiment is net-positive in
r/rust (mixed but enthusiastic) and strongly positive in
r/systems_programming. r/cscareerquestions shows the widest variance and
is listed but de-prioritized.
```

5.6 Sentiment as a Runtime Signal

Community sentiment is not currently run as a formal offline evaluation track (there is no eval/sentiment_eval.py), but it is exercised through unit tests in tests/test_sentiment.py that validate the label-to-signed-score mapping and the per-subreddit aggregation. At runtime, sentiment is displayed through a sentiment bar whose width is the absolute value of the score clamped to one and whose color is

green for scores at least 0.5, amber for scores in [0.2, 0.5), and red for scores below 0.2. These thresholds are visualization heuristics, not calibrated toxicity decisions, and the report consistently frames the sentiment output as a community-tone signal on retrieved corpus passages rather than as a real-time moderation-risk detector.

5.7 Confidence and Threats to Validity

The retrieval evaluation uses a proxy query (the concatenation of ground-truth subreddit names) rather than the real runtime input, which means the eventual Recall@K and MRR numbers will be optimistic about the retriever in isolation and pessimistic about the full online pipeline, which gets an additional theme-extraction and query-rewrite stage at inference. The generation evaluation is single-rater on a feasible-sized sample of reports (fifty per the original plan) without inter-rater agreement reporting, a known limitation of the current framework. Both gaps are flagged in the evaluation documentation as intended next-step improvements.

6 Conclusions

CreatorPal demonstrates that the combination of offline corpus construction, hybrid dense-plus-sparse retrieval, LLM-driven query expansion, cross-encoder reranking, community-level sentiment, and a grounded generation layer can deliver a production-quality creator-audience-matching product on a single Reddit snapshot and a single locally hosted LLM. The eight-stage pipeline produces a structurally valid payload even when optional stages are unavailable, the retrieval stack is evaluated under Recall@K and MRR on held-out ground-truth pairs, and the frontend renders the result behind a strict adapter contract that converts backend failure into a styled error rather than a traceback. The most important takeaway from building the system is architectural: graceful degradation is not a cosmetic concern but a core correctness property of a multi-model RAG pipeline, and the _try_init guard together with the adapter boundary give us a system that is safe to run in demo, safe to run in offline-mock mode, and safe to run in production without changing a single line of the orchestration.

If given more time, three directions are most likely to pay off. First, add a dedicated offline sentiment evaluation script with a labeled sample of subreddit text so that the currently runtime-only sentiment signal can be benchmarked under a fixed test set, and so that the threshold choices in the UI can be calibrated rather than chosen heuristically. Second, extend the retrieval evaluator to replay the full online pipeline (user query plus theme extraction plus query rewriting plus HyDE plus rerank) rather than the current ground-truth-proxy query, so that Recall@K and MRR reflect what the user actually experiences. Third, add a multi-rater generation evaluation with inter-rater agreement reporting, and grow the evaluation sample beyond the current feasible-scope fifty queries. Beyond these immediate steps, personalization (learning creator-specific weightings over the hybrid fusion coefficients), real-time comment integration (so that community tone tracks the present rather than the April 2019 snapshot), and explicit modeling of subreddit posting rules (to turn the strategy report from community-tone guidance into a compliance-aware recommendation) are all natural extensions of the current architecture.

7 Team Contributions

Per the course requirement that each final report include a contributions section denoting what each member worked on, Table 2 lists each member's primary area and core deliverables, followed by per-member narratives that document specific modules and key technical decisions. The source of truth for this section is the repository's commit history; every file listed below was committed by the attributed member.

Member	Primary Area	Core Deliverable
Runxin Shao	Retrieval + Backend Integration	src/retrieval/, src/pipeline.py, src/config.py, YouTube ingest, report generator
Ziqi Yang	Frontend + Deployment + Documentation	app/, Dockerfile, docker-startup, doc/, data/read_zst.py
Gaoyuan Shi	Data Pipeline	data/build_*.py, data/preprocess_corpus.py
Mingkai Gao	Analytics + Evaluation + Project Initialization	src/pal/, src/sentiment/, eval/, tests/, repository structure

Table 2. Team contributions summary.

7.1 Runxin Shao — Retrieval Pipeline + Backend Integration

Runxin owned the entire retrieval stack under src/retrieval/ (faiss_search.py, bm25_search.py, hybrid_search.py, query_rewriter.py, reranker.py, hyde.py, theme_extractor.py), the YouTube ingest module (src/ingest/youtube.py), the RAG-conditioned augmented generator (src/generator/augmented_gen.py), the frozen Settings dataclass (src/config.py) loading fourteen environment variables with typed defaults, and the primary authorship of CreatorPalPipeline in src/pipeline.py implementing the eight-stage end-to-end orchestration with graceful skip of optional components. He also contributed tests/test_retrieval_smoke.py for BM25, FAISS, and hybrid fusion. Key technical decisions: the _try_init guard that allows the pipeline to degrade gracefully rather than abort when optional components are unavailable; the 0.85 / 0.15 FAISS / BM25 weight split paired with min-max normalization so that score scales are comparable before fusion; the append-only HyDE merge policy that ensures query expansion can only add recall and never demote a strong first-pass candidate; and a metadata-loader fix that allows both JSON-array and NDJSON formats to be consumed interchangeably across BM25 and FAISS.

7.2 Ziqi Yang — Frontend + Deployment + Project Documentation

Ziqi owned the full Streamlit interface (app/streamlit_app.py) including the idle hero, dashboard layout, light-and-dark CSS theme, and entrance animations; the strict payload adapter (app/helpers/adapter.py) with type validation, URL normalization, and sentinel-safe coercion; the pure HTML component builders (app/helpers/components.py) for metric cards, subreddit rows, and sentiment bars; the deterministic contract-compliant mock pipeline (app/helpers/mock_pipeline.py) for offline development and demo; the streaming CLI reader for Reddit .zst JSONL dumps (data/read_zst.py); the application container image

(Dockerfile) and the single-command deploy script (docker-startup); and the entire doc/ tree (backend pipeline, retrieval, corpus references; four frontend specification documents; and project-level contributing.md, team.md, and README.md). Key technical decisions: the strict adapter boundary that prevents the frontend from rendering unvalidated data and converts pipeline failure into structured error cards rather than tracebacks; the mock-pipeline fallback that keeps the UI demo-able regardless of backend availability; direct CSS injection through st.markdown with unsafe_allow_html for full theme control without forking Streamlit internals; and the two-state page machine backed by st.session_state that provides clean navigation without page reloads.

7.3 Gaoyuan Shi — Data Pipeline

Gaoyuan owned the offline data pipeline: the streaming .zst to NDJSON reducer (data/build_reddit_slim.py) keeping seven fields with optional score-filter and post-limit controls; the YouTube-to-subreddit ground-truth pair extractor (data/build_ground_truth.py) with regex URL detection, channel-ID inference, and CSV output; the profile chunker and encoder (data/build_faiss_index.py) with sixty-four-token sliding windows, sixteen-token overlap, all-mpnet-base-v2 encoding, and a FAISS IndexFlatIP writer; and the subreddit profile aggregator (data/preprocess_corpus.py, co-implemented with Runxin Shao) using heapq.nlargest for O(n log k) top-fifty selection and a configurable min-posts filter. Key technical decisions: the use of streaming .zst decompression throughout the pipeline so that the 15 GB dump never has to be fully materialized in memory; the O(n log k) heap aggregation instead of per-subreddit full sort; the sixty-four / sixteen window-and-overlap choice to balance chunk granularity against index size; and the idempotent output-existence check in each build script so the pipeline is safely re-runnable after partial failure.

7.4 Mingkai Gao — Analytics + Evaluation + Project Initialization

Mingkai owned the analytics and evaluation modules: the PAL executor (src/pal/executor.py) combining LLM code generation with RestrictedPython sandboxed execution; the sentiment analyzer (src/sentiment/analyzer.py) implementing the RoBERTa classifier with signed per-subreddit score aggregation; the retrieval evaluator (eval/retrieval_eval.py) with Recall@K and MRR metrics over the ground-truth YouTube–subreddit pairs; the generation quality framework (eval/generation_eval.py) covering coherence, grounding, and actionability; and the associated unit tests (tests/test_pal.py, tests/test_sentiment.py, tests/test_eval.py). He also initialized the repository and executed the structural migration from the original ReAct / Gemini architecture to the current RAG-centric pipeline, establishing the module boundaries on which the rest of the team built. Key technical decisions: the use of RestrictedPython.compile_restricted with a curated safe_builtins allowlist (pandas, numpy, and standard math functions) to permit analytics expressiveness while blocking arbitrary code execution; the mapping of RoBERTa three-class output onto a signed [-1, +1] scale so that sentiment scores can be averaged and thresholded meaningfully; and the repository-wide module boundary layout that made the parallel four-person development possible in the first place.

References

[1] S. E. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333-389, 2009.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. EMNLP, Hong Kong, 2019, pp. 3982-3992. https://arxiv.org/abs/1908.10084

[3] J. Johnson, M. Douze, and H. Jegou, "Billion-Scale Similarity Search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535-547, 2021. https://arxiv.org/abs/1702.08734

[4] L. Gao, X. Ma, J. Lin, and J. Callan, "Precise Zero-Shot Dense Retrieval without Relevance Labels," in Proc. ACL, Toronto, 2023, pp. 1762-1777. https://arxiv.org/abs/2212.10496

[5] X. Ma, Y. Gong, P. He, H. Zhao, and N. Duan, "Query Rewriting for Retrieval-Augmented Large Language Models," in Proc. EMNLP, Singapore, 2023. https://arxiv.org/abs/2305.14283

[6] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Liang, and G. Neubig, "PAL: Program-aided Language Models," in Proc. ICML, Honolulu, 2023. https://arxiv.org/abs/2211.10435

[7] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," arXiv preprint, 2019. https://arxiv.org/abs/1901.04085

[8] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," arXiv preprint, 2019. https://arxiv.org/abs/1907.11692